

|                                                                                                                                                                |                                                                                                                                                         |                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
|  <b>ISEL</b><br><small>INSTITUTO SUPERIOR DE<br/>ENGENHARIA DE LISBOA</small> | <b>Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores</b><br><b>Computação Distribuída (Época Recurso)</b><br><b>MEIC, MEIM</b> |                   |
| <b>Nº</b>                                                                                                                                                      | <b>Nome:</b>                                                                                                                                            | <b>16/02/2022</b> |

**EXAME ÉPOCA RECURSO (1ª PARTE, SEM CONSULTA, DURAÇÃO: 45 min., 11 valores)**

Nas questões de 1 a 5 assinale as afirmações como verdadeira (V), falsa (F) ou não responde (≡). Uma opção assinalada corretamente soma 0,25 valores e incorretamente desconta 0,125 valores (50% da cotação de cada alínea).

**1. [1 val.] Sobre as características dos sistemas distribuídos**

|                          |                                                                                                                                                                                   |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | O desenvolvimento de componentes <i>stateless</i> facilita a implementação de escalabilidade horizontal                                                                           |
| <input type="checkbox"/> | A propriedade de transparência à concorrência apenas se aplica nas componentes do lado do servidor                                                                                |
| <input type="checkbox"/> | O suporte a falhas de partições de rede, na arquitetura de um sistema, promove maior disponibilidade                                                                              |
| <input type="checkbox"/> | Para que um sistema tenha elasticidade é preciso haver forma de monitorizar a carga da aplicação (acessos correntes, % CPU, etc.) o que aumenta a complexidade do desenvolvimento |

- 2. [1 val.]** Um sistema distribuído, constituído por 4 processos (p0,p1,p2,p3), utiliza comunicação por *multicast* e um mecanismo baseado em relógios vetoriais que garante ordenação causal de mensagens. O estado dos relógios locais é: **p0=[7,2,5,1]** ; **p1=[7,2,4,1]** ; **p2=[6,2,5,1]** ; **p3=[7,2,5,2]**. Considere que **p0** envia uma mensagem *multicast* (**m**), com o respetivo vetor **Vm=[8,2,5,1]**.

|                          |                                                                                                     |
|--------------------------|-----------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | A mensagem <b>m</b> terá de ficar retida ( <i>hold</i> ) em todos os recetores p1, p2, p3           |
| <input type="checkbox"/> | A mensagem <b>m</b> é retida em p2 porque falta receber primeiro uma mensagem anterior de p0        |
| <input type="checkbox"/> | A mensagem <b>m</b> é retida em p1 pois a mensagem de p0 tem uma dependência causal com outra de p2 |
| <input type="checkbox"/> | A mensagem <b>m</b> é retida em p3 porque o relógio de p0 tem um atraso em relação ao relógio de p3 |

**3. [1 val.] Relativamente ao desenvolvimento de aplicações em gRPC**

|                          |                                                                                                                                                                              |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | Não faz sentido usar um <i>stub</i> bloqueante para chamar uma operação que usa <i>stream</i> de cliente                                                                     |
| <input type="checkbox"/> | Numa operação com <i>stream</i> de cliente e <i>stream</i> de servidor em simultâneo, a cada pedido do cliente corresponde forçosamente uma mensagem de resposta do servidor |
| <input type="checkbox"/> | Uma aplicação servidora gRPC pode implementar vários serviços com contratos <i>protobuf</i> diferentes                                                                       |
| <input type="checkbox"/> | Num <i>StreamObserver</i> , a chamada do método <i>onNext</i> implica chamar logo a seguir o método <i>onCompleted</i>                                                       |

**4. [1 val.] Relativamente ao sistema de mensagens RabbitMQ**

|                          |                                                                                                                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | Os consumidores ( <i>subscribers</i> ) ligam-se a <i>exchanges</i> para receberem mensagens                                                                                             |
| <input type="checkbox"/> | Um <i>exchange</i> do tipo <i>direct</i> com múltiplas <i>queues</i> associadas ( <i>bind</i> ), entrega cada mensagem a todas as <i>queues</i> independentemente da <i>binding key</i> |
| <input type="checkbox"/> | Uma <i>queue</i> associada a um <i>exchange</i> do tipo <i>topic</i> pode conter mensagens com diferentes <i>strings</i> de encaminhamento ( <i>routing keys</i> )                      |
| <input type="checkbox"/> | Os produtores ( <i>producers</i> ) ligam-se a <i>exchanges</i> para enviar mensagens                                                                                                    |

**5. [1 val.] Sobre algoritmos de exclusão mútua, eleição e consenso**

|                          |                                                                                                                                                                                                                         |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | Quando se usa <i>Spread Toolkit</i> a eleição de um <i>Leader</i> ou Coordenador é garantida por um <i>daemon</i> que controla os grupos de participantes.                                                              |
| <input type="checkbox"/> | O algoritmo de exclusão mútua Ricart&Agrawala, ao usar relógios lógicos de Lamport, garante a ordenação ( <i>Happened-before</i> ) no acesso ao recurso crítico.                                                        |
| <input type="checkbox"/> | Nos algoritmos Raft e Bully, o participante <i>Leader</i> é sempre o que tiver o maior identificador.                                                                                                                   |
| <input type="checkbox"/> | No algoritmo Raft, a responsabilidade de coordenar a atualização das réplicas dos <i>logs</i> , é realizada por um servidor <i>Leader</i> que é eleito para um determinado período de tempo designado por <i>Term</i> . |

6. [2 val.] Apresente o essencial do contrato *protobuf* de um servidor gRPC que implementa uma operação gRPC de nome **exame** como se indica a seguir:

```
public StreamObserver<Query> exame(StreamObserver<Result> responseObserver) {  
    return new StreamObserverQueries(responseObserver);  
}  
  
public class StreamObserverQueries implements StreamObserver<Query> {  
    StreamObserver<Result> result;  
    public StreamObserverQueries(StreamObserver<Result> result) { this.result=result; }  
    @Override  
    public void onNext(Query query) {  
        if (query.getNum() % 2 == 0)  
            result.onNext(Result.newBuilder().setIsPar(true).build());  
    }  
    @Override  
    public void onError(Throwable throwable) { }  
    @Override  
    public void onCompleted() { result.onCompleted(); }  
}
```

Operação:

Mensagens:

7. [2 val.] Considere uma aplicação de troca de mensagens, entre utilizadores, baseada no *middleware* RabbitMQ, em que se podem enviar mensagens sobre 3 assuntos diferentes (ex: Cultura, Política, Desporto). Cada utilizador pode receber, em simultâneo, mensagens de 1 ou mais assuntos. Descreva sucintamente e apresente um diagrama que concretize o cenário descrito.

8. [2 val.] No sistema de *containers* Docker, considere que existe uma imagem disponibilizada no repositório DockerHub de nome `cd/exameApp:v1`. **A)** É possível alterar o conteúdo dessa imagem? **B)** É possível usar essa imagem para criar (*build*) uma nova imagem? Justifique as suas respostas.